Lexique officiel OPAL-RT (terminologies de référence)

Cette page est le **lexique de référence** pour les termes utilisés dans les pages qualité, SDLC, spécification, validation et tests. Son objectif est de fournir une source commune de vocabulaire afin d'éviter les contradictions, les définitions dispersées et les débats causés par des mots utilisés à plusieurs sauces.

Comment utiliser cette page. Lorsqu'un terme est utilisé dans une autre page, il devrait être aligné avec la définition ici. Si une page a besoin d'un détail opérationnel, elle peut le préciser localement, mais sans redéfinir le terme d'une manière contradictoire.

Attention aux mots piégés. Certains mots de l'industrie sont ambigus : « unitaire », « intégration », « E2E », « système », « acceptation », « couverture », « environnement », « smoke », « sanity », etc. Ici, on privilégie des définitions explicites : ce qui est observé, à quel niveau, avec quelle interface, selon quel angle, dans quel contexte et pour quelle preuve.

Convention linguistique. Pour préserver l'uniformité dans un contexte très anglophone, le terme principal est présenté en français lorsque c'est possible, suivi du terme anglais entre parenthèses. Les acronymes ou noms d'usage très établis peuvent être conservés, mais leur sens français doit être explicite dans la définition ou la clarification.

Navigation. Utilisez la table des matières de Confluence ou la section « Recherche rapide par famille » ci-dessous pour accéder rapidement aux familles de termes. Le lexique est volontairement centralisé : il sert de source de vérité terminologique pour les pages qualité, SDLC, spécification, validation et tests.

Recherche rapide par famille

- **Fondations SDLC** — cycle de vie de réalisation logicielle (Software Development Life Cycle), éliciter (elicit), définir (define), planifier (plan), implémenter (implement), diffuser (diffuse), confirmer (confirm), faire du sens (SENSE), forger (FORGE), veiller (CARE).
- **Spécification** — besoin (need), signal (signal), exigence (requirement), spécification (specification), spécification produit (product specification), spécification exécutable (executable specification), spécification atomique (atomic specification), spécification IF... THEN..., règle métier (business rule), cas (case), scénario (scenario), requête (request / query), spécification par l'exemple (specification by example), traçabilité (traceability).

- **Validation et tests** — test (test), validation (validation), vérification (verification), preuve de validation (validation evidence), conformité (compliance), acceptation (acceptance), non-régression.
- **Axes de classification des tests** — niveau d'intégration (integration level), interface observée (observed interface), angle d'observation (observation angle), type de validation (validation type), technique (technique), moment ou intention (timing or intent), contexte d'exécution (execution context), montage (test setup).
- **Niveaux d'intégration** — petit (small), moyen (medium), grand (large), bout en bout (end-to-end / E2E).
- **Interfaces observées** — interface utilisateur (user interface / UI), interface applicative (application programming interface / API), interface machine (machine interface).
- **Angles d'observation** — boîte noire (black-box), boîte grise (gray-box), boîte blanche (white-box).
- **Famille « intégrer »** — intégration (integration), intégré (integrated), intégral (full), intégralité (completeness), intégrité (integrity), intègre (intact / ethical).
- **Types et catégories de validation** — fonctionnel (functional), performance (performance), sécurité (security), résilience (resilience), accessibilité (accessibility), contrat (contract), intégrité des données (data integrity), compatibilité (compatibility), installation (installation), observabilité (observability), etc.
- **Montages et doubles** — double de test (test double), simulacre (mock), bouchon (stub), faux fonctionnel (fake), espion (spy), simulateur (simulator), émulateur (emulator), dépendance réelle (real dependency), HIL, SIL, MIL, PIL.
- **Machine, simulation et temps réel** — modèle dans la boucle (MIL), logiciel dans la boucle (SIL), processeur dans la boucle (PIL), matériel dans la boucle (HIL), temps réel (real-time), gigue (jitter), latence (latency), débit (throughput), échéance temporelle (deadline), pire cas d'exécution (WCET).
- **Livraison, gouvernance et qualité** — définition de prêt (DoR), définition de terminé (DoD), définition de vérité / Definition of True (DoT), décision Go / No-Go, dérogation (waiver), diffusion (release), build candidat (release candidate build), retour arrière (rollback), qualimétrie (quality metrics / qualimetry), KPI, SLO, SLI, SLA, MTTR.
- **Pratiques CI/CD et opérations** — intégration continue (continuous integration), livraison continue (continuous delivery), déploiement continu (continuous deployment), pipeline CI/CD, environnement, journal (log), trace (trace), métrique (metric), incident (incident), post-mortem.
- **Rôles et responsabilités** — VP R&D, directeur architecte, directeur qualité, ingénieur du flux de valeur (VSE), responsable produit (PM), propriétaire du produit (PO), chargé de livraison

(CdL), responsable technique (Tech Lead), responsable d'équipe (Team Lead), architecte, architecte qualité, spécialiste qualité, coach qualité, vérificateur-validateur indépendant (IV&V), automaticien de tests, qualimétriste, auditeur qualité, ingénieur qualité / spécialiste qualité (QA), opérations (Ops), spécialiste métier (SME), partie prenante (stakeholder), RTE, Scrum Master, coach agile, gestionnaire de programme, gestionnaire de projet, gestionnaire de changement, rôle compensatoire, rôle catalyseur.

- **Termes complémentaires utiles** — carnet de produit (backlog), récit utilisateur (user story), épopée (epic), fonctionnalité (feature), périmètre (scope), artefact (artifact), livrable (deliverable), dette technique (technical debt), dépendance (dependency), environnement (environment), observabilité (observability).
- **Incidents, défauts et support** — défaut (defect), anomalie (anomaly), incident (incident), problème (problem), ticket de support (support ticket), sévérité (severity), priorité (priority), contournement (workaround), correctif (fix), analyse de cause racine (RCA), post-mortem, escalade (escalation).

Fondations SDLC

Terme	Définition de référence	À ne pas confondre avec
Cycle de vie de réalisation logicielle (Software Development Life Cycle / SDLC)	Cycle structurant le passage du besoin vers la confirmation terrain. Chez Opal-RT : éliciter → définir → planifier → implémenter → diffuser → confirmer.	Un simple workflow Jira, une méthode agile ou une liste de cérémonies.
Éliciter (elicit)	Phase où l'on recueille, comprend, qualifie et priorise les besoins bruts, signaux, incidents, demandes et observations avant de formaliser une solution.	Définir. Éliciter sert à comprendre le problème ; définir sert à formaliser la solution attendue.
Définir (define)	Phase où les besoins compris sont transformés en spécifications atomiques, complètes, testables, modélisées, traçables et approuvées.	Coder, planifier ou seulement écrire un récit utilisateur vague.

Planifier (plan)	Phase où l'on organise le périmètre, les dépendances, les risques, le découpage, les intégrations, les tests, l'automatisation, la livraison, les métriques et les responsabilités avant l'exécution.	Mettre des tickets dans un sprint seulement.
Implémenter (implement)	Phase où l'on traduit fidèlement les spécifications en code, tests, intégrations, documentation et preuves de conformité.	Coder sans test, sans documentation ou sans lien avec la spécification.
Diffuser (diffuse)	Phase où l'on assemble, valide, package, déploie, documente, transfère et décide formellement si une version peut être rendue disponible.	La diffusion (release) comme artefact ou livraison. Ici, *diffuser* nomme l'action du SDLC ; *release* peut aussi désigner la version livrée.
Confirmer (confirm)	Phase où l'on recueille les preuves terrain, observations, métriques, incidents, retours utilisateurs et signaux réels après diffusion.	Une validation pré-livraison. Confirmer observe la réalité d'usage.
Faire du sens (SENSE)	Domaine du SDLC où l'on comprend les besoins, clarifie, spécifie et réduit l'ambiguïté avant l'exécution.	Un rôle unique ou une équipe isolée.

Forger (FORGE)	Domaine du SDLC où l'on planifie, implémente, intègre, teste, assemble et prépare la livraison. « Forger » met l'accent sur la transformation concrète des spécifications en produit fonctionnel, robuste et livrable.	Coder seulement.
Veiller (CARE)	Domaine du SDLC où l'on diffuse, supporte, exploite, observe le terrain, confirme et alimente l'amélioration continue.	Support réactif seulement après incident.

Spécification

⚠ **Note de convergence — spécification.** Les termes historiques comme critère d'acceptation, condition de succès, exigence, test, cas de test, scénario, Gherkin, BDD, AAA ou « As a / I want / So that » ne doivent plus être utilisés comme couches séparées de définition du produit. Dans la convention cible, ils convergent vers une seule nomenclature : la **spécification IF... THEN...**. L'objectif n'est pas d'écrire des spécifications puis de recréer ensuite des critères, scénarios ou cas sous un autre nom ; ce serait seulement ajouter de la complexité et multiplier les langages pour dire la même chose.

Une spécification demeure la définition unique du comportement attendu. Les anciens termes peuvent être compris comme des héritages, des formats d'outil ou des habitudes de langage, mais la source de vérité doit rester la spécification conditionnelle : **IF** contexte, condition, état, entrée ou événement ; **THEN** comportement, résultat, contrainte, effet ou signal attendu observable.

Terme	Définition de référence	Clarification importante
Besoin (need)	Problème, objectif, résultat attendu, contrainte ou opportunité exprimée ou observée avant d'être formalisée en spécification.	Un besoin n'est pas automatiquement une solution ni une spécification prête à implémenter.

Signal (signal)	Indice factuel ou qualitatif qui révèle un besoin, un risque, une tendance ou une opportunité : incident, métrique, retour client, observation terrain, dette, régression, demande récurrente.	Un signal doit être interprété ; il ne devient pas automatiquement une spécification.
Exigence (requirement)	Ancienne appellation d'une attente à satisfaire. Dans la convention cible, une exigence bien formulée converge vers une spécification IF… THEN….	Convergence : ne pas maintenir une exigence comme couche séparée. Si elle définit un comportement attendu, elle doit être reformulée en spécification IF… THEN….
Spécification (specification)	Définition unique et normative d'un comportement attendu sous forme de condition : une spécification commence toujours par **IF** et contient toujours un **THEN**. Elle décrit précisément : **IF** une condition, un contexte, une entrée, un état ou un événement est vrai, **THEN** un comportement, résultat, contrainte, effet ou signal attendu doit être observable. Dans un SDLC moderne, la spécification est le langage commun qui relie portfolio, produit, architecture, développement, QA, opérations, support et gouvernance. Elle peut exister à tous les niveaux, du	Attention : ne pas utiliser le mot « spécification » pour autre chose que cette définition conditionnelle IF… THEN…. L'objectif n'est plus d'avoir plusieurs langages parallèles — test, cas, scénario, exigence, règle, mapping — mais une nomenclature commune capable de traverser code, outils de test, documentation, diagrammes, modélisation, pipelines, opérations et IA.

	comportement atomique au produit complet, et couvrir dépendances, environnements, configurations, profils, requêtes, données, modèles, interfaces et preuves attendues.	
Spécification produit (product specification)	Ensemble cohérent de spécifications IF… THEN… décrivant le produit ou une capacité produit dans son entier : comportements, règles, interfaces, dépendances, contraintes, données, validations, opérations, environnements, configurations, profils, paramètres, preuves et décisions.	Elle évite que portfolio, développement et opérations utilisent des nomenclatures différentes pour parler du même produit. Les validations et tests qui décrivent le comportement attendu font partie de cette définition produit.
Spécification exécutable (executable specification)	Spécification IF… THEN… exprimée dans une forme assez structurée pour être exécutée, vérifiée, générée, automatisée ou utilisée comme instruction précise par des outils, pipelines ou systèmes d'IA.	C'est un changement majeur : la spécification devient à la fois contrat de compréhension, source d'automatisation et instruction fiable pour l'IA.
Forme conditionnelle universelle (IF… THEN…)	Format obligatoire et transversal d'une spécification : **IF** une condition, un contexte, une entrée, un état ou un événement est vrai, **THEN** un comportement, résultat, contrainte, effet ou	Cette forme remplace les syntaxes spécialisées comme AAA, Gherkin ou « As a / I want / So that » lorsqu'on veut une nomenclature universelle. Elle est compréhensible par l'humain, exploitable par la machine, compatible avec le

	signal attendu doit être observable.	code, les modèles, les validations et les instructions d'IA.
SPECIFY	Artefact ou ensemble structuré de spécifications IF… THEN… utilisé comme source de vérité pour le périmètre, l'implémentation, l'intégration, la validation, l'automatisation, l'IA et la traçabilité.	SPECIFY n'est pas une documentation séparée pour un seul rôle ; c'est une source partagée et compatible avec code, diagrammes, modèles, pipelines et décisions.
Spécification atomique (atomic specification)	Spécification IF… THEN… assez petite pour être comprise, codée, intégrée, automatisée et validée clairement, mais assez complète pour porter une valeur ou une preuve vérifiable.	Atomique ne veut pas dire trivial ; cela veut dire divisible, clair et vérifiable. Elle peut correspondre à ce qu'on appelait auparavant un test unitaire, une règle, un contrat, une assertion, une requête ou un critère précis.
Critère d'acceptation (acceptance criterion)	Ancienne appellation d'une condition utilisée pour décider si un élément est acceptable. Dans la convention cible, un critère d'acceptation est une spécification IF… THEN….	Convergence : ne pas écrire des spécifications puis recréer des critères d'acceptation séparés. Le critère devient la spécification, exprimée en IF… THEN….
Règle métier (business rule)	Contrainte, décision, calcul, politique ou invariant du domaine métier. Dans la convention cible, une règle métier s'exprime comme une spécification IF… THEN….	Convergence : la règle métier ne doit pas devenir une couche parallèle. Si elle définit un comportement attendu, elle est une spécification.
Cas (case)	Ancienne appellation d'une situation particulière utilisée pour illustrer ou exercer une condition. Dans la convention cible, le cas est absorbé par la	Convergence : ne pas maintenir des « cas » comme langage séparé. Ils servent seulement à rendre une spécification concrète.

	spécification IF… THEN… ou par ses données d'exemple.	
Scénario (scenario)	Ancienne appellation d'un déroulement d'événements ou d'actions. Dans la convention cible, un scénario décrivant un comportement attendu converge vers une ou plusieurs spécifications IF… THEN….	Convergence : ne pas utiliser le scénario comme couche de définition parallèle. La source de vérité demeure la spécification conditionnelle.
Requête (request / query)	Demande structurée adressée à une interface, un service, une base, un modèle ou un système pour obtenir, modifier, déclencher ou observer une information ou un comportement.	Une requête devient un élément de spécification lorsqu'elle décrit précisément l'entrée, le contexte, la réponse, les erreurs et les contraintes attendues sous forme IF… THEN….
Spécification par l'exemple (specification by example)	Technique qui utilise des exemples concrets pour clarifier une spécification IF… THEN… et rendre visibles les variantes, limites et données importantes.	Convergence : les exemples doivent soutenir la spécification, pas recréer un langage séparé comme Gherkin, BDD ou cas de test.
Traçabilité (traceability)	Capacité à relier besoin → spécification → implémentation → validation → résultats → décision → retour terrain.	La traçabilité devient beaucoup plus forte lorsque la spécification sert de point de rassemblement plutôt que de simple document isolé.

Validation et tests — concepts de base

Terme	Définition de référence	Mythe ou confusion à éviter
Test (test)	Ancienne appellation d'une manière structurée d'obtenir de l'information sur un produit,	Convergence : ne pas créer une couche « tests » séparée des spécifications. Le test

	un comportement, un contrat, une intégration, une hypothèse ou un risque. Dans la convention cible, un test qui décrit un comportement attendu est une spécification exécutable IF… THEN….	devient l'exécution, l'automatisation ou la preuve d'une spécification.
Validation (validation)	Action de confirmer qu'un comportement, produit, système ou livrable répond à l'usage, aux besoins et aux spécifications attendues.	Ne pas réduire la validation à la fin du projet ; elle existe à chaque phase et s'appuie sur les spécifications.
Vérification (verification)	Action de confirmer qu'un artefact respecte une règle, une spécification, une norme ou un contrat défini.	Vérifier que c'est conforme ne prouve pas toujours que c'est utile ou suffisant.
Preuve de validation (validation evidence)	Résultat, rapport, trace, observation, mesure, capture, journal, métrique ou approbation démontrant qu'une spécification a été validée et ce que la validation a conclu.	Une preuve doit être reliée à la spécification qu'elle prouve réellement.
Conformité (compliance)	Alignement avec une spécification, norme, politique, contrat, standard ou contrainte définie.	La conformité n'est pas automatiquement la qualité complète ; elle prouve l'alignement à un référentiel.
Acceptation (acceptance)	Décision indiquant qu'un livrable satisfait les spécifications attendues par une partie prenante, un client, un PO, un utilisateur ou un contrat.	Convergence : l'acceptation ne repose pas sur une couche séparée de critères ; elle repose sur des spécifications IF… THEN… satisfaites.

Non-régression (non-regression)	Intention de confirmer qu'un changement n'a pas invalidé des spécifications qui étaient déjà vraies.	Convergence : la non-régression est un ensemble de spécifications qui doivent rester vraies, pas une catégorie isolée déconnectée du produit.

Axes de classification des spécifications

Axe	Définition	Exemples
Niveau d'intégration	Périmètre réel assemblé pour spécifier, observer ou valider un comportement.	Petit (small), moyen (medium), grand (large).
Interface observée	Frontière par laquelle une spécification stimule ou observe le système.	Interface utilisateur, interface applicative, interface machine.
Angle d'observation	Degré de visibilité sur l'intérieur du système pendant la spécification, l'exécution ou la validation.	Boîte noire (black-box), boîte grise (gray-box), boîte blanche (white-box).
Type de validation	Propriété, risque ou qualité que la spécification cherche à démontrer ou rendre vérifiable.	Fonctionnel, performance, sécurité, résilience, accessibilité, contrat.
Technique	Moyen utilisé pour obtenir l'information ou produire la preuve associée à une spécification.	Exploratoire, mutation, par propriétés, analyse statique, injection de fautes.
Moment ou intention	Pourquoi ou quand une spécification est exécutée, vérifiée ou utilisée comme preuve.	Fumée (smoke), cohérence ciblée (sanity), revalidation, post-déploiement, diffusion, alpha, bêta.

Contexte d’exécution	Où et dans quelles conditions la spécification s’exécute ou se vérifie.	Local, CI, staging, pré-production, production, HIL, SIL.
Montage	Composition des dépendances réelles, simulées ou remplacées nécessaires pour exécuter ou valider la spécification.	Simulacres (mocks), bouchons (stubs), simulateurs, dépendances réelles, hybride réel/simulé.

Niveaux d’intégration

Terme	Définition de référence	Image simplifiée
Petit (small)	Validation du plus petit comportement isolable et testable : fonction, règle, composant UI, service local, module logique, script, driver ou élément machine isolé.	A ou B isolé.
Moyen (medium)	Validation de l’intégration entre plusieurs éléments petits : contrat, échange, appel, mapping, orchestration partielle ou dépendance contrôlée.	A → B.
Grand (large)	Validation d’un ensemble complet orienté usage, mission ou résultat observable, avec un flux significatif de bout en bout selon une frontière définie.	A → B → A.

Bout en bout (end-to-end / E2E)	Validation d'un flux complet entre un point de départ et un point d'arrivée explicitement définis, avec résultat observable.	Le terme est utile seulement si les frontières et dépendances sont nommées.

Interfaces observées

Terme	Définition de référence	Exemples
Interface utilisateur / UI	Frontière visible ou interactive par laquelle un utilisateur agit ou observe le système.	Écran, formulaire, bouton, navigation, message, composant visuel.
Interface applicative / API	Frontière logicielle par laquelle des composants, services ou systèmes échangent des opérations, données ou événements.	API REST, service, fonction publique, événement, file de messages, contrat applicatif.
Interface machine	Frontière où le logiciel interagit avec matériel, système, processus, drivers, signaux, simulation, temps réel ou ressources machine.	Driver, capteur, actionneur, bus, protocole matériel, processeur cible, simulation HIL.

Angles d'observation

Terme	Définition de référence	Clarification
Boîte noire (black-box)	Validation qui observe les entrées, sorties et effets visibles sans s'appuyer sur l'implémentation interne.	Boîte noire ne veut pas dire grand ; on peut faire du boîte noire en petit.
Boîte grise (gray-box)	Validation qui observe ou exploite certains éléments internes sans piloter ou	Utile pour l'intégration, les diagnostics, les contrats et les états intermédiaires.

	connaître toute l'implémentation.	
Boîte blanche (white-box)	Validation qui utilise la connaissance de la structure interne, du code, des branches, états, mécanismes ou instruments internes.	Boîte blanche ne veut pas dire unitaire ; c'est un angle d'observation.

Famille « intégrer »

Terme	Définition de référence	Exemple de confusion
Intégration	Action d'assembler des éléments distincts et de valider que leur collaboration produit le comportement attendu.	Ne veut pas simplement dire installer dans un environnement.
Intégré	Inséré, branché, inclus ou injecté dans un ensemble plus large.	Un élément intégré n'est pas automatiquement correctement intégré.
Intégralité	Totalité, ensemble complet de quelque chose.	Valider l'intégralité d'une liste n'est pas la même chose que valider l'intégration entre les parties.
Intégrité	État d'une donnée, structure, système ou artefact qui demeure conforme, cohérent, complet et non corrompu.	L'intégrité est une propriété à valider, pas un niveau d'intégration.
Intégral	Complet, entier, sans partie manquante.	Un test intégral ne précise pas automatiquement s'il est small, medium ou large.
Intègre	Pour une donnée ou un artefact : non altéré et cohérent. Pour une personne : honnête, éthique.	À ne pas confondre avec « intégré » ou « intégration ».

Types et catégories de validation — définitions courtes

Terme	Définition de référence	Note
Fonctionnel (functional)	Spécification IF… THEN… qui décrit un comportement produit attendu selon une fonction, une règle, une capacité observable ou un comportement attendu.	Convergence : ne pas maintenir une validation fonctionnelle comme langage séparé ; elle s'exprime en spécifications IF… THEN…. Peut être UI, API ou machine ; petit, moyen ou grand.
Règles métier (business rules)	Spécifications IF… THEN… décrivant les décisions, calculs, contraintes ou politiques propres au domaine métier.	Convergence : une règle métier est une spécification lorsqu'elle définit un comportement attendu. Les exemples ou tables de décision doivent soutenir les spécifications, pas devenir une couche parallèle.
Contrat (contract)	Ensemble de spécifications IF… THEN… décrivant l'accord explicite d'une interface : champs, types, erreurs, statuts, événements, délais, sémantique et comportements attendus.	Convergence : un contrat est un ensemble de spécifications d'interface, pas une nomenclature séparée. Le contrat peut être technique, fonctionnel ou opérationnel.
API (API)	Spécifications IF… THEN… appliquées à une interface applicative : requêtes, réponses, authentification, erreurs, pagination, idempotence, performance ou contrat.	Convergence : une validation API doit décrire le comportement attendu sous forme IF… THEN…. Une API n'est pas automatiquement grand format.
Intégrité des données (data integrity)	Spécifications IF… THEN… vérifiant que les données	À distinguer de l'intégration.

	restent cohérentes, complètes, valides et non corrompues.	
Migration (migration)	Spécifications IF… THEN… vérifiant qu'un changement de version, modèle, plateforme ou base conserve le sens et l'utilisabilité des données ou comportements.	Un script qui termine sans erreur ne prouve pas une migration réussie.
Limite / valeurs frontières (boundary values)	Spécifications IF… THEN… décrivant le comportement attendu aux bornes : minimums, maximums, seuils, tailles, valeurs extrêmes, transitions.	Les défauts apparaissent souvent aux frontières.
Robustesse (robustness)	Spécifications IF… THEN… décrivant comment le système doit supporter des entrées, conditions ou contextes imparfaits sans comportement dangereux ou incohérent.	Proche de la résilience, mais centrée sur la tenue face à l'imperfection.
Résilience (resilience)	Spécifications IF… THEN… décrivant la capacité à continuer, se dégrader proprement, récupérer ou tolérer une perturbation.	Ce n'est pas « ne jamais tomber » ; c'est aussi tomber et revenir correctement.
Performance (performance)	Spécifications IF… THEN… portant sur temps de réponse, latence, débit, ressources, stabilité, capacité, déterminisme ou tenue sous charge.	Convergence : un critère de performance s'exprime en IF… THEN…, par exemple : IF charge X THEN latence ≤ Y. La performance peut être petit, moyen ou grand.
Charge (load)	Spécifications IF… THEN… décrivant le comportement	À distinguer du stress qui dépasse les conditions normales.

	attendu sous une charge attendue ou représentative.	
Stress (stress)	Spécifications IF… THEN… qui poussent au-delà des conditions normales pour identifier limites, points de rupture et modes de défaillance.	Doit être contrôlé et observable.
Endurance / tenue longue durée (soak)	Spécifications IF… THEN… décrivant la stabilité attendue sur une durée prolongée : fuites, dérives, accumulation, ralentissement.	Important pour les dégradations lentes.
Sécurité (security)	Spécifications IF… THEN… décrivant la protection attendue contre abus, attaques, erreurs de configuration, accès non autorisés, vulnérabilités et fuites.	Convergence : un critère de sécurité s'exprime en IF… THEN…. Ne se limite pas au pentest de fin de projet.
Confidentialité (confidentiality)	Spécifications IF… THEN… décrivant que seules les personnes ou systèmes autorisés peuvent accéder aux données sensibles.	Liée à la sécurité et à la protection des données.
Sûreté (safety)	Spécifications IF… THEN… décrivant comment le système doit éviter les états dangereux pour personnes, équipements, opérations ou environnement.	La sûreté (safety) n'est pas la sécurité (security).
Compatibilité (compatibility)	Spécifications IF… THEN… décrivant que le comportement reste correct dans les contextes supportés :	Convergence : un critère de compatibilité s'exprime en IF… THEN…. La matrice supportée doit être explicite.

	OS, navigateurs, versions, matériels, dépendances, formats.	
Configuration (configuration)	Spécifications IF… THEN… décrivant les comportements attendus selon options, paramètres, profils ou feature flags.	Inclut interactions entre paramètres.
Installation (installation)	Spécifications IF… THEN… décrivant l'installation attendue du produit dans un contexte supporté.	Inclut prérequis, permissions et configuration initiale.
Déploiement (deployment)	Spécifications IF… THEN… décrivant comment le produit doit être livré et rendu disponible correctement dans un environnement cible.	L'environnement n'est pas le niveau de spécification.
Retour arrière (rollback)	Spécifications IF… THEN… décrivant comment revenir à un état antérieur fiable après un changement, déploiement ou incident.	Un retour arrière doit être spécifié et vérifié, pas seulement prévu.
Interface utilisateur (user interface / UI)	Spécifications IF… THEN… appliquées à l'interface utilisateur : rendu, états, interactions, navigation, messages et comportements visibles.	UI ne veut pas dire E2E.
Expérience utilisateur (user experience / UX)	Spécifications IF… THEN… ou observations structurées portant sur l'expérience globale : compréhension, fluidité, confiance, efficacité, friction et satisfaction.	Dépasse le simple fonctionnement des boutons.

Utilisabilité (usability)	Spécifications IF… THEN… décrivant qu'un utilisateur cible peut accomplir sa tâche efficacement, clairement et avec un effort raisonnable.	Liée à UX, mais centrée sur la capacité d'usage.
Accessibilité (accessibility)	Spécifications IF… THEN… décrivant que le produit est utilisable par des personnes ayant différentes capacités, outils ou contraintes.	La conformité WCAG aide, mais ne remplace pas toujours l'usage réel.
Observabilité (observability)	Spécifications IF… THEN… décrivant les signaux que le système doit exposer pour comprendre, surveiller et diagnostiquer : journaux, métriques, traces, événements.	Convergence : un critère d'observabilité s'exprime en IF… THEN…. Un système non observable devient difficile à supporter.
Supportabilité (supportability)	Spécifications IF… THEN… décrivant que le support peut comprendre, reproduire, diagnostiquer et aider à résoudre les problèmes.	À prévoir avant diffusion, pas après incident.
Maintenabilité (maintainability)	Spécifications IF… THEN… ou critères structurés portant sur la facilité à comprendre, modifier, vérifier et faire évoluer le système.	Inclut code, architecture, documentation et dette.
Conformité réglementaire (regulatory compliance)	Spécifications IF… THEN… décrivant le respect attendu de lois, règlements, normes ou obligations applicables.	Doit être traçable et vérifiable.

Techniques, moments et intentions

Terme	Nature	Définition de référence

Exploratoire (exploratory testing)	Technique	Validation guidée par l'apprentissage, l'observation et l'adaptation, idéalement avec charte, notes et résultats traçables.
Ad hoc (ad hoc testing)	Technique / intention	Vérification ponctuelle et peu formalisée. Utile pour apprendre rapidement, mais insuffisante comme stratégie reproductible.
Par propriétés (property-based testing)	Technique	Génère de nombreux cas pour vérifier des propriétés générales qui doivent rester vraies.
Mutation (mutation testing)	Technique	Modifie artificiellement le code pour vérifier si les tests détectent les changements, mesurant la force des assertions.
Analyse statique (static analysis)	Technique	Analyse un artefact sans l'exécuter : code, configuration, dépendances, modèles ou documentation.
Injection de fautes (fault injection)	Technique	Introduit volontairement des erreurs contrôlées pour valider la réaction du système.
Ingénierie du chaos (chaos engineering)	Stratégie / technique	Introduit des perturbations contrôlées pour apprendre sur la résilience d'un système, avec hypothèse, garde-fous et observabilité.
Test fumée (smoke testing)	Moment / intention	Vérification rapide des capacités critiques pour savoir

		si une version est assez stable pour continuer.
Test de cohérence ciblée (sanity testing)	Moment / intention	Vérification ciblée après un changement pour confirmer que le comportement semble cohérent dans la zone touchée.
Retest (re-test)	Moment / intention	Réexécution d'une validation après correction ou changement pour confirmer le point corrigé.
Confirmation de correctif (fix verification)	Moment / intention	Validation qu'un défaut corrigé ne se reproduit plus dans le cas concerné.
Post-déploiement (post-deployment validation)	Moment / contexte	Validation après livraison pour confirmer que les capacités attendues sont disponibles dans le contexte déployé.
Alpha (alpha testing)	Moment	Validation précoce par un groupe limité, souvent interne ou proche de l'équipe.
Bêta (beta testing)	Moment	Validation par des utilisateurs réels ou représentatifs avant disponibilité générale.

Montages et doubles de test

Terme	Définition de référence	Clarification
Double de test (test double)	Remplacement contrôlé d'une dépendance réelle pour faciliter une validation.	Terme générique : simulacre, bouchon, faux fonctionnel, espion, simulateur, émulateur.
Simulacre (mock)	Double souvent utilisé pour vérifier des interactions	Trop utiliser des simulacres peut tester l'implémentation

	attendues : appels, paramètres, ordre ou nombre d'appels.	plutôt que le comportement utile.
Bouchon (stub)	Double qui fournit des réponses prédéfinies pour contrôler un contexte.	Un bouchon ne prouve pas que la dépendance réelle agit pareil.
Faux fonctionnel (fake)	Implémentation simplifiée mais fonctionnelle d'une dépendance, souvent utilisée pour les tests.	Plus réaliste qu'un bouchon simple, mais pas nécessairement représentatif du réel.
Espion (spy)	Double ou instrument qui enregistre ce qui s'est passé pour permettre une vérification.	Utile pour observer sans toujours remplacer complètement.
Simulateur (simulator)	Représentation contrôlée d'un système, environnement, machine ou comportement réel.	Sa valeur dépend de sa fidélité et de sa calibration.
Émulateur (emulator)	Reproduction d'un environnement ou matériel permettant d'exécuter un comportement comme s'il était sur la cible.	À distinguer d'un simulateur, qui modélise plutôt un comportement.
Dépendance réelle (real dependency)	Service, base, matériel, réseau ou système véritable utilisé pendant la validation.	Augmente la représentativité, mais peut augmenter coût, lenteur et instabilité.
Hybride réel/simulé (real/simulated hybrid)	Montage combinant parties réelles et parties simulées pour équilibrer confiance, coût et contrôle.	Il faut expliciter ce qui est réel et ce qui ne l'est pas.

Machine, simulation et temps réel

Terme	Définition de référence	Clarification

Modèle dans la boucle (Model-in-the-loop / MIL)	Validation d'un modèle dans une boucle simulée, avant ou indépendamment du code final.	Stratégie d'intégration, pas garantie de qualité à elle seule.
Logiciel dans la boucle (Software-in-the-loop / SIL)	Validation du logiciel dans un environnement simulé.	Dépend de la fidélité du simulateur.
Processeur dans la boucle (Processor-in-the-loop / PIL)	Validation où le code s'exécute sur le processeur cible ou représentatif, avec une partie de l'environnement simulée.	Permet d'observer les contraintes de cible sans tout le système réel.
Matériel dans la boucle (Hardware-in-the-loop / HIL)	Validation utilisant du matériel réel dans une boucle avec simulation ou contrôle.	Très représentatif, mais ne remplace pas les validations petit et moyen.
Temps réel (real-time)	Contexte où la correction dépend aussi du respect de contraintes temporelles, pas seulement du résultat logique.	La moyenne ne suffit pas ; les pires cas et les échéances temporelles comptent.
Déterminisme temporel (temporal determinism)	Capacité à produire un comportement temporel prévisible, répétable et dans les contraintes requises.	Essentiel en simulation temps réel et systèmes critiques.
Gigue (jitter)	Variation temporelle autour d'un délai ou intervalle attendu.	Importante en temps réel, réseau, audio/vidéo et acquisition.
Latence (latency)	Délai entre une entrée, une demande, un signal ou un événement et la réponse ou l'effet observable correspondant.	À distinguer du débit : un système peut traiter beaucoup d'éléments, mais répondre trop lentement.
Débit (throughput)	Quantité d'éléments, requêtes, événements, données ou opérations traités par unité de temps.	Doit être interprété avec la latence, l'utilisation des ressources et la stabilité.

Échéance temporelle (deadline)	Limite de temps à respecter pour qu'un traitement, signal ou comportement demeure correct.	En temps réel, manquer une échéance peut être un défaut même si le résultat logique est bon.
Pire cas d'exécution (Worst-Case Execution Time / WCET)	Durée maximale attendue ou démontrée pour exécuter une opération dans des conditions définies.	Plus pertinent qu'une moyenne lorsqu'une garantie temporelle est requise.
Temps réel dur (hard real-time)	Contexte où le non-respect d'une contrainte temporelle constitue une défaillance critique.	La tolérance aux dépassements est nulle ou très limitée selon le risque.
Temps réel souple (soft real-time)	Contexte où le non-respect occasionnel d'une contrainte temporelle dégrade la qualité ou l'expérience sans constituer nécessairement une défaillance critique.	Les seuils d'acceptabilité doivent tout de même être explicites.

Livraison, gouvernance et qualité

Terme	Définition de référence	Clarification
Définition de prêt (Definition of Ready / DoR)	Dans ce lexique, la DoR est définie du point de vue du développement et de l'implémentation : critères qui doivent être satisfaits avant qu'un élément de backlog, une story, une tâche ou une spécification puisse entrer en implémentation, souvent au début ou juste avant un sprint.	La DoR est le côté gauche du flux d'implémentation : elle protège l'équipe contre le démarrage sur ambiguïté. Un élément prêt possède un périmètre clair, des spécifications compréhensibles, les dépendances critiques identifiées, les risques visibles, les conditions minimales de validation connues et assez de

		contexte pour commencer sans deviner.
Définition de terminé (Definition of Done / DoD)	Dans ce lexique, la DoD est définie du point de vue du développement pendant l'implémentation, souvent dans le cadre d'un sprint : critères qui doivent être satisfaits pour considérer que le travail de développement est réellement terminé, intégré, vérifié, documenté et traçable selon le niveau de qualité attendu.	La DoD se vit pendant le Just-in-Time (JiT) de l'implémentation : elle transforme l'intention en incrément livrable. « Presque fini » n'est pas terminé. La DoD confirme que le travail produit est complet : code, spécifications, validations, preuves, documentation, revue, intégration et traçabilité pertinentes sont disponibles.
Définition de vrai (Definition of True / DoT)	Dans ce lexique, la DoT est définie du point de vue du développement à la sortie de l'implémentation : critères permettant d'affirmer que ce qui est livré, validé, mesuré ou déclaré est factuellement vrai, vérifiable et aligné avec la réalité observée.	La DoT est le côté droit du flux d'implémentation : elle protège contre les validations déclaratives, les preuves non sourcées et les conclusions non démontrées. Un élément est « vrai » seulement si la source est connue, la preuve est accessible, le contexte est explicite, la mesure est reproductible ou expliquée, et la conclusion correspond à ce qui a réellement été observé. Même si DoR et DoT peuvent sembler incluses dans une DoD large, l'usage Scrum a progressivement forcé une lecture en trois étapes : DoR avant, DoD pendant, DoT après.
Décision Go / No-Go (Go / No-Go decision)	Décision formelle d'autoriser ou non une diffusion, fondée	Un Go n'est pas un acte de foi.

	sur des preuves, seuils, risques et responsabilités.	
Dérogation (waiver)	Exception formellement documentée lorsqu'une règle ou exigence ne peut pas être respectée, avec justification, risque, durée et plan de retour.	Une dérogation récurrente est un signal d'amélioration, pas une nouvelle norme implicite.
Build candidat (release candidate build)	Artefact assemblé, versionné, traçable et prêt à être validé pour une diffusion potentielle.	Ce n'est pas un simple zip de fichiers.
Diffusion (release)	Version ou livraison planifiée rendue disponible à une audience définie, avec périmètre, artefacts, notes, validation et décision.	À distinguer de l'action « diffuser (diffuse) » dans le SDLC, ainsi que d'un build, d'un sprint ou d'une branche.
Retour arrière (rollback)	Retour vers un état antérieur connu et acceptable après changement ou incident.	Doit être planifié et testé.
Qualimétrie (quality metrics / qualimetry)	Ensemble de métriques, mesures, signaux et indicateurs utilisés pour observer, piloter et améliorer continuellement la qualité, peu importe la façon de mesurer ou la famille de métriques utilisée. La qualimétrie couvre autant la santé du produit que la santé des processus, du cadre de travail, du flux de livraison, des opérations, de la stabilité, de la conformité, du risque, de l'expérience utilisateur, de l'efficacité et du retour sur investissement.	Le terme est moins officialisé en français que certains équivalents anglais, mais il est utile ici pour nommer l'approche globale d'amélioration continue par indicateurs. La qualimétrie n'est pas une métrique unique ni un tableau de bord décoratif : c'est un système de lecture de la santé et de la valeur. Les métriques assistent le jugement, les décisions et l'amélioration ; elles ne les remplacent pas.

Indicateur clé de performance (Key Performance Indicator / KPI)	Mesure utilisée pour suivre l'atteinte d'un objectif important, souvent liée à une décision ou à un seuil de pilotage.	Un KPI doit être actionnable ; sinon, il devient seulement décoratif.
Objectif de niveau de service (Service Level Objective / SLO)	Cible mesurable de service à atteindre sur une période donnée, par exemple disponibilité, latence ou taux d'erreur.	Un SLO exprime l'objectif ; il doit être alimenté par des indicateurs mesurables.
Indicateur de niveau de service (Service Level Indicator / SLI)	Mesure concrète utilisée pour observer un aspect du service, par exemple disponibilité réelle, latence p95 ou taux d'échec.	Un SLI mesure ; un SLO fixe la cible.
Entente de niveau de service (Service Level Agreement / SLA)	Engagement formel de service, souvent contractuel, qui précise les niveaux attendus et parfois les conséquences d'un non-respect.	Un SLA ne devrait pas être inventé sans capacité réelle de mesure et de respect.
Temps moyen de rétablissement (Mean Time To Recovery / MTTR)	Temps moyen nécessaire pour rétablir un service ou une capacité après incident ou défaillance.	À interpréter avec la sévérité, la fréquence et la qualité du rétablissement.
Fréquence de déploiement (deployment frequency)	Mesure de la fréquence à laquelle des changements sont déployés ou rendus disponibles dans un contexte défini.	Plus fréquent n'est pas automatiquement meilleur si la qualité ou la stabilité diminue.
Délai de changement (lead time for change)	Temps entre le début d'un changement et sa disponibilité dans le contexte cible défini.	Le point de départ et le point d'arrivée doivent être explicités.

Taux d'échec des changements (change failure rate)	Proportion de changements qui causent un incident, une dégradation, un retour arrière, une correction urgente ou une intervention non planifiée.	Doit inclure une définition claire de ce qui compte comme échec.
Couverture de code (code coverage)	Mesure des parties du code exécutées par des tests.	Ne prouve pas la qualité des assertions ni la couverture des risques.
Couverture des spécifications (specifications coverage)	Mesure ou preuve que les spécifications sont associées à des validations et résultats.	Couvert ne veut pas dire suffisamment bien validé.

Mythes et termes à risque

- **« Unitaire »** — Peut désigner une unité de code, un composant UI, un style, une habitude ou un outil. Dans ce lexique, on préfère préciser le niveau réel : petit, moyen ou grand, ainsi que l'interface et le montage.
- **« E2E »** — Doit toujours préciser ses frontières : départ, arrivée, dépendances réelles ou simulées, résultat observable. Sans cela, le terme devient trop vague.
- **« Intégration »** — Ne signifie pas automatiquement grand, pré-production, pipeline ou environnement. Il faut nommer les éléments réellement assemblés.
- **« Système »** — Doit préciser la frontière du système observé. Un « test système » peut couvrir un sous-système significatif ou un système complet selon le contexte.
- **« Environnement »** — Local, CI, staging, pré-production et production décrivent où l'on exécute, pas le niveau d'intégration.
- **« Pyramide de tests »** — Sert surtout à discuter du coût, de la quantité et de la maintenabilité des automatisations. Elle ne définit pas à elle seule les niveaux d'intégration, les branches, les environnements ou la livraison.
- **« Couverture »** — Indique ce qui est exercé ou relié, mais pas automatiquement la pertinence, la force des assertions ni la suffisance par rapport aux risques.
- **« QA »** — La validation n'appartient pas seulement au QA. Elle concerne produit, développement, architecture, opérations, support, sécurité, utilisateurs et gouvernance.
- **« AAA »** — Arrange / Act / Assert est une structure historique utile dans certains codes, mais ne doit pas devenir une nomenclature produit parallèle. Dans la convention cible, le comportement attendu converge vers une spécification IF… THEN….

- **« Gherkin »** — Given / When / Then est un format d'outil et d'habitude BDD. Il peut être traduit, mais la source de vérité cible demeure IF… THEN… pour éviter de multiplier les langages.
- **« BDD »** — Behavior-Driven Development reste une pratique de collaboration possible, mais les scénarios BDD ne doivent pas recréer une couche séparée. Le comportement attendu converge vers la spécification IF… THEN….
- **« TDD »** — Test-Driven Development peut guider l'implémentation, mais les tests produits doivent être compris comme des spécifications exécutables lorsqu'ils décrivent un comportement attendu.
- **« As a / I want / So that »** — Ce format peut aider à exprimer une intention utilisateur, mais il ne remplace pas la spécification du comportement attendu. La définition normative demeure IF… THEN….
- **« Cas de test », « scénario de test » et « conditions de succès »** — Ces termes historiques ne doivent pas devenir des couches additionnelles après la spécification. Ils convergent vers une seule source de vérité : la spécification IF… THEN….

Rôles et responsabilités

Cette section définit les rôles souvent référencés dans les pages SDLC, contrat qualité, validation et gouvernance. Elle demeure volontairement générale : elle décrit la contribution typique et le sens du rôle, sans fixer les tâches détaillées, l'imputabilité exacte ou les responsabilités opérationnelles. Ces éléments devront être définis et validés dans la matrice RACI officielle. Le lexique sert donc à éviter les confusions de vocabulaire, pas à remplacer le RACI.

Les rôles ci-dessous incluent ceux qui sont actifs chez Opal-RT ainsi que des rôles courants dans l'industrie (SAFe, Scrum, ITIL, ingénierie systèmes, etc.). Même si certains ne sont pas formellement présents dans l'organisation, les connaître aide à comprendre la littérature, les cadres de travail et les échanges avec les partenaires.

Note Québec — usage du mot « ingénieur ». Au Québec, le titre « ingénieur » est réservé aux membres de l'Ordre des ingénieurs du Québec (OIQ). Les termes de l'industrie traduits depuis l'anglais — par exemple *engineer*, *systems engineer*, *site reliability engineer* ou *value stream engineer* — doivent donc être compris ici comme des libellés de référence ou d'équivalence terminologique, pas comme l'attribution d'un titre professionnel réservé. Lorsque le contexte québécois l'exige, on peut privilégier des formulations comme « spécialiste », « responsable technique », « technicien », « technique », « analyste » ou « rôle d'ingénierie » selon le poste réel et les règles internes applicables.

Principe de gouvernance — rôles et maturité. Il ne faut pas chercher à remplir tous les rôles. Il faut d'abord comprendre les tâches et les responsabilités, puis les clarifier dans le RACI avant de distribuer les rôles en fonction de la capacité et de la maturité de l'organisation. Certains rôles peuvent être combinés, d'autres temporaires ou catalyseurs, et d'autres encore ne sont pas nécessaires à un stade donné. Multiplier les rôles sans clarifier les responsabilités crée des rôles compensatoires, ce qui génère souvent plus de problèmes que cela n'en règle.

Terme	Définition de référence	Clarification
VP R&D (Vice-président recherche et développement)	Rôle de direction qui définit la vision technologique, les priorités de recherche et développement, l'allocation des ressources et la gouvernance technique à l'échelle de l'organisation.	Le VP R&D porte la responsabilité stratégique du portefeuille technologique ; il ne porte pas la livraison opérationnelle au quotidien.
Directeur architecte (Director of Architecture)	Rôle de direction qui assure la cohérence architecturale à travers les produits, plateformes et équipes. Il définit les standards techniques, les patterns de référence et arbitre les décisions structurantes.	Le directeur architecte opère au niveau stratégique ; l'architecte logiciel opère généralement au niveau du produit, du système ou du composant.
Directeur qualité (Director of Quality)	Rôle de direction qui porte la vision, la stratégie et la	Le directeur qualité fixe le cap ; l'architecte qualité conçoit la

	gouvernance qualité à l'échelle de l'organisation : standards, contrat qualité, qualimétrie, formation et amélioration continue.	stratégie d'exécution ; le coach qualité accompagne l'adoption ; les spécialistes qualité produisent les preuves.
Ingénieur du flux de valeur (Value Stream Engineer / VSE)	Rôle associé à la santé globale d'un flux de valeur produit : cohérence, arbitrages, risques, qualité livrée et décisions majeures lorsque requises.	Le VSE ne remplace pas PM, PO, développement ou QA ; il porte l'imputabilité du flux et des décisions majeures. Au Québec, le libellé « engineer » doit être interprété avec la note sur le titre réservé.
Responsable produit (Product Manager / PM)	Rôle généralement associé à la vision produit, au marché, aux clients, à la feuille de route, à la valeur et aux priorités d'affaires.	Le PM répond surtout au quoi et au pourquoi. Il ne gère pas nécessairement le carnet quotidien de l'équipe.
Propriétaire du produit (Product Owner / PO)	Rôle généralement associé au carnet de produit tactique, au découpage en éléments réalisables, aux spécifications IF… THEN… et à la priorisation avec l'équipe.	Le PO répond surtout au comment découper et dans quel ordre réaliser. Il ne remplace pas la vision marché du PM.
Chargé de livraison (Delivery Manager / CdL)	Rôle généralement associé au suivi du SDLC, au flux de livraison, à la préparation des décisions, aux dépendances, aux preuves et à la coordination de diffusion. Il agit comme coordinateur et gardien du cycle de développement logiciel complet, de l'élicitation des besoins jusqu'à la confirmation	Le CdL prépare et orchestre ; il ne décide pas seul du contenu produit ni de la valeur métier. Il s'assure que les critères de sortie de chaque phase SDLC sont validés, que les dépendances inter-équipes sont coordonnées, que les risques sont visibles, que le plan de release est réaliste et communiqué, que les métriques et signaux de santé

	post-livraison et la boucle d'amélioration continue.	sont disponibles, que le build candidat respecte le contrat qualité, que le comité Go / No-Go est préparé, que le handover vers CARE est complété, que le support est prêt et que les retours post-livraison retournent vers le backlog d'élicitation. Il n'écrit pas le code, n'exécute pas les validations à la place des spécialistes, ne rédige pas les spécifications, ne comble pas les trous organisationnels, ne décide pas seul du Go / No-Go, ne fait pas le support client et ne déploie pas en production.
Développeur (developer)	Rôle qui implémente les spécifications en code, tests, intégrations, documentation technique et preuves de conformité.	Développer ne veut pas dire seulement coder ; cela inclut comprendre, tester, intégrer, documenter et contribuer à la qualité.
Responsable technique (Technical Lead / Tech Lead)	Rôle qui guide les décisions techniques d'une équipe ou d'un périmètre : conception, cohérence du code, intégration, dette technique et risques techniques.	Le Tech Lead n'est pas automatiquement le gestionnaire hiérarchique. Il porte surtout le leadership technique, alors que le Team Lead porte davantage la dynamique d'équipe, la coordination humaine ou la gestion selon l'organisation.
Responsable d'équipe (Team Lead)	Rôle qui soutient la coordination d'une équipe, la dynamique de collaboration, le suivi des personnes, les	À ne pas confondre avec Tech Lead : le Team Lead n'est pas nécessairement l'autorité technique principale, et le

	priorités opérationnelles et parfois la gestion hiérarchique selon l'organisation.	Tech Lead n'est pas nécessairement responsable des personnes.
Architecte (architect)	Rôle qui veille à la cohérence systémique : frontières, contrats, dépendances, évolutivité, maintenabilité, risques transversaux et alignement avec la vision technique.	L'architecture ne se limite pas aux diagrammes ; elle doit rendre les intégrations et compromis compréhensibles et validables.
Ingénieur qualité / spécialiste qualité / testeur / ingénierie de validation (Quality Assurance / QA)	Rôle qui aide à définir, concevoir, exécuter, automatiser, analyser et améliorer les validations nécessaires pour obtenir des preuves proportionnelles aux risques.	La qualité n'appartient pas seulement au QA ; le QA rend la preuve, les risques et les trous de couverture plus visibles. Au Québec, utiliser « spécialiste qualité », « analyste qualité » ou « technicien qualité » lorsque le titre professionnel d'ingénieur n'est pas applicable.
Architecte qualité (Quality Architect)	Rôle qui conçoit la stratégie qualité à l'échelle du produit, du programme ou du flux de valeur : standards, contrats qualité, couverture, traçabilité, risques, preuves et architecture de validation.	Travaille en duo avec l'architecte pour rendre la qualité conçue dès le départ, pas ajoutée en fin de parcours. Ne remplace pas les spécialistes qualité ; il oriente le système qualité global.
Développement et opérations (DevOps)	Principe et pratique visant à intégrer développement, opérations, automatisation, livraison, observabilité et amélioration continue.	DevOps n'est pas seulement une équipe, un outil CI/CD ou une personne qui maintient des pipelines.
Opérations (operations / Ops)	Rôle ou discipline responsable de l'exploitation, disponibilité, déploiement, surveillance,	Les opérations doivent être considérées avant la livraison,

	incidents, capacité et stabilité des environnements ou services.	pas seulement après un incident.
Support / Veiller (support / CARE)	Rôle ou domaine qui reçoit les signaux terrain, aide les utilisateurs, documente les incidents, soutient le diagnostic et alimente l'amélioration continue.	Le support n'est pas seulement réactif ; il est une source critique de signaux pour éliciter et confirmer.
Utilisateur (user)	Personne ou groupe qui utilise réellement le produit ou système pour accomplir une tâche, atteindre un résultat ou opérer un contexte donné.	L'utilisateur n'est pas toujours l'acheteur, le client ou le décideur.
Client (customer)	Personne ou organisation qui achète, finance, commande ou reçoit officiellement la valeur livrée.	Le client peut être différent de l'utilisateur final ; les deux perspectives doivent être distinguées.
Spécialiste métier (Subject Matter Expert / SME)	Personne qui possède une expertise profonde d'un domaine, d'un usage, d'un produit, d'un système ou d'une contrainte spécialisée.	Le spécialiste aide à valider le sens et la justesse, mais ne remplace pas à lui seul la traçabilité ni les preuves.
Partie prenante (stakeholder)	Personne, équipe ou organisation affectée par une décision, une spécification, une livraison, un risque ou un changement.	Toutes les parties prenantes n'ont pas le même pouvoir de décision ni le même niveau d'implication.
Comité Go / No-Go (Go / No-Go committee)	Groupe qui prépare ou soutient une décision formelle de diffusion à partir des preuves, risques, seuils, dérogations et responsabilités définies.	Le comité ne doit pas compenser une absence de preuves ; il rend la décision explicite et traçable.

Ingénieur du train de livraison (Release Train Engineer / RTE)	Rôle SAFe qui facilite et coordonne un train de livraison agile : flux, dépendances, cérémonies, risques et amélioration continue à l'échelle.	Le RTE ne gère pas le contenu produit ; il veille à la fluidité du flux de livraison inter-équipes. Au Québec, le mot « ingénieur » doit être interprété comme équivalence terminologique de l'anglais.
Scrum Master / facilitateur agile	Rôle qui facilite les cérémonies, protège l'équipe des perturbations, aide à lever les obstacles et favorise l'amélioration continue du cadre de travail.	Le Scrum Master n'est pas un gestionnaire de projet ni un chef d'équipe hiérarchique.
Coach agile (Agile Coach)	Rôle qui accompagne les équipes, les gestionnaires et l'organisation dans l'adoption, la maturation et l'amélioration des pratiques agiles.	Le coach agile ne dirige pas les équipes ; il développe leur autonomie et leurs pratiques.
Gestionnaire de programme (Program Manager)	Rôle qui coordonne plusieurs projets, flux de valeur ou initiatives liées pour atteindre un objectif stratégique commun.	À ne pas confondre avec gestionnaire de projet, qui couvre généralement un seul projet défini.
Gestionnaire de projet (Project Manager)	Rôle qui planifie, suit et coordonne les activités, le budget, l'échéancier, les risques et les livrables d'un projet défini.	Dans un contexte agile, certaines responsabilités peuvent être distribuées entre PO, CdL, Scrum Master, Team Lead ou autres rôles.
Gestionnaire de changement (Change Manager)	Rôle qui accompagne l'adoption des changements organisationnels, culturels ou processus par les personnes affectées.	Le changement technique et le changement organisationnel sont deux choses distinctes, même s'ils peuvent être liés.

Analyste d’affaires (Business Analyst / BA)	Rôle qui analyse les besoins, les processus et les données d’affaires pour formuler des recommandations ou des spécifications.	Dans la convention cible, les spécifications produites convergent vers IF… THEN….
Concepteur UX/UI (UX/UI Designer)	Rôle qui conçoit l’expérience utilisateur, les interfaces, les parcours et l’utilisabilité du produit.	Le design UX/UI influence directement les spécifications d’interface, d’accessibilité et d’expérience.
Rédacteur technique (Technical Writer)	Rôle qui produit, organise et maintient la documentation technique destinée aux utilisateurs, développeurs, opérations ou partenaires.	La documentation fait partie du livrable ; elle n’est pas un ajout optionnel après coup.
Spécialiste de fiabilité de site (Site Reliability Engineer / SRE)	Rôle qui applique les principes d’ingénierie logicielle à l’exploitation, la fiabilité, l’observabilité, la capacité et l’automatisation des opérations.	L’approche SRE est complémentaire à DevOps ; elle met l’accent sur les SLO, budgets d’erreur et l’automatisation. Au Québec, « spécialiste de fiabilité de site » peut être préférable au titre « ingénieur ».
Spécialiste en ingénierie systèmes (Systems Engineer)	Rôle qui veille à la cohérence globale d’un système composé de logiciel, matériel, modèles, interfaces et contraintes opérationnelles.	Particulièrement pertinent en simulation, temps réel et HIL. Au Québec, choisir le libellé de poste selon le titre professionnel réellement applicable.
Intégrateur système (System Integrator)	Rôle qui assemble, configure et valide l’interopérabilité entre composants, sous-systèmes ou systèmes provenant de sources différentes.	L’intégration système ne se limite pas au code ; elle couvre aussi matériel, réseaux, configurations et données.

Responsable de la sécurité (Security Champion / Security Engineer)	Rôle qui intègre la sécurité dans le SDLC : analyse de menaces, revue de code sécurité, spécifications de sécurité et réponse aux vulnérabilités.	La sécurité ne doit pas être une activité de fin de projet ; le Security Champion l'ancre dans le quotidien.
Gestionnaire de configuration (Configuration Manager)	Rôle qui assure l'identification, le contrôle, le suivi et la traçabilité des artefacts, versions, configurations et baselines.	Particulièrement pertinent pour les produits avec de nombreuses variantes, plateformes cibles ou dépendances matérielles.
Rôle compensatoire (anti-pattern)	Rôle créé avec le temps pour compenser une lacune organisationnelle : RACI flou, responsabilité non assumée, capacité absente ou processus insuffisamment défini.	Un rôle compensatoire se reconnaît souvent lorsqu'on ne peut pas expliquer sa raison d'être sans décrire un dysfonctionnement. La solution n'est pas de multiplier les rôles, mais de clarifier les tâches et responsabilités d'abord, puis de distribuer les rôles en conséquence.
Rôle catalyseur (rôle temporaire)	Rôle introduit volontairement et temporairement pour accélérer une transformation, développer une compétence ou combler un écart de maturité.	Contrairement au rôle compensatoire, le rôle catalyseur a une mission définie, un horizon de fin et un critère de succès. Il devrait disparaître ou se transformer lorsque l'organisation a internalisé la capacité.

Termes complémentaires utiles

Terme	Définition de référence	Clarification
Carnet de produit (product backlog / backlog)	Ensemble ordonné d'éléments à clarifier, prioriser, planifier ou	Un carnet de produit n'est pas une poubelle à demandes ; il

	réaliser pour un produit, service ou flux de valeur.	doit être entretenu, priorisé et relié à la valeur.
Récit utilisateur (user story)	Découpage réalisable d'un besoin ou d'une spécification, destiné à être compris, planifié, implémenté et validé par une équipe.	Un récit utilisateur n'est pas automatiquement une spécification complète ; il doit pointer vers le comportement attendu et ses spécifications.
Épopée (epic)	Grand ensemble de valeur ou de capacité regroupant plusieurs fonctionnalités, récits utilisateurs ou spécifications liées.	Une épopée ne doit pas servir à cacher un périmètre flou ; elle doit être progressivement clarifiée.
Fonctionnalité (feature)	Capacité produit significative et livrable, généralement découpée en récits utilisateurs ou spécifications atomiques.	Une fonctionnalité doit porter une valeur observable, pas seulement un regroupement technique arbitraire.
Périmètre (scope)	Frontière explicite de ce qui est inclus, exclu, observé, modifié ou validé.	Beaucoup de confusions viennent d'un périmètre implicite : toujours nommer la frontière.
Artefact (artifact)	Élément produit ou utilisé dans le SDLC : spécification, modèle, code, build, rapport, preuve, configuration, package, note de livraison.	Un artefact doit idéalement être versionné, traçable et relié à son usage.
Livrable (deliverable)	Artefact ou résultat remis à une partie prenante, un client, une équipe ou un environnement selon un objectif défini.	Un livrable n'est pas toujours du code ; cela peut être une spécification, une preuve ou une documentation.
Dette technique (technical debt)	Écart connu entre l'état actuel et un état souhaitable de qualité, maintenabilité, architecture, automatisation ou	La dette n'est pas toujours mauvaise si elle est consciente, documentée, priorisée et remboursable.

	documentation, qui crée un coût futur.	
Risque (risk)	Événement, incertitude ou condition pouvant affecter valeur, qualité, coût, délai, sécurité, conformité, supportabilité ou livraison.	Un risque doit idéalement avoir impact, probabilité, mitigation et propriétaire.
Atténuation (mitigation)	Action visant à réduire la probabilité, l'impact ou l'exposition d'un risque.	Une atténuation ne supprime pas toujours le risque ; elle le rend acceptable ou contrôlé.
Dépendance (dependency)	Élément, équipe, service, composant, décision, environnement ou fournisseur nécessaire à la réalisation, validation ou livraison.	Les dépendances implicites sont une source majeure de surprises en intégration.
Contrat d'intégration (integration contract)	Accord explicite entre éléments qui doivent collaborer : interface, données, protocole, erreurs, responsabilités, délais, versions et comportements attendus.	Un contrat d'intégration peut être validé en medium avant d'attendre un large complet.
Correspondance (mapping)	Correspondance entre deux représentations : champs, unités, formats, statuts, modèles de données ou concepts métier.	Une correspondance peut être une source de défauts critiques même si chaque côté fonctionne isolément.
Versionnement (versioning)	Pratique qui identifie et distingue les versions d'un artefact, produit, API, modèle, dépendance ou documentation.	Versionner ne suffit pas ; il faut aussi savoir quoi est compatible avec quoi.
Branche (branch)	Ligne de développement dans un système de contrôle de	Une branche n'est pas un environnement, une diffusion

	version.	ou un niveau de test.
Pipeline CI/CD (CI/CD pipeline)	Chaîne automatisée qui construit, vérifie, assemble, teste, package, publie ou déploie des artefacts selon des règles définies.	Un pipeline exécute des validations ; il ne définit pas à lui seul la stratégie qualité.
Environnement (environment)	Contexte d'exécution ou de validation : local, CI, test, staging, pré-production, production, cible matérielle, banc HIL, etc.	L'environnement décrit où l'on exécute ; il ne décrit pas automatiquement ce qu'on valide.
Observabilité (observability)	Capacité d'un système à fournir des signaux permettant de comprendre son état et son comportement : journaux, métriques, traces, événements.	Sans observabilité, les validations grand format et les diagnostics terrain deviennent beaucoup plus fragiles.
Journal (log)	Enregistrement d'événements ou d'informations produits par un système pendant son exécution.	Un journal utile doit être exploitable, corrélé, compréhensible et proportionné.
Métrique (metric)	Mesure quantitative utilisée pour observer un comportement, une qualité, une tendance, un seuil ou un risque.	Une métrique sans contexte peut être trompeuse.
Trace (trace)	Suivi corrélé d'un flux ou d'une requête à travers plusieurs composants, services ou étapes.	Essentiel pour diagnostiquer les validations moyennes et grandes distribuées.

Incidents, défauts et support

Terme	Définition de référence	Clarification

Défaut (defect)	Écart entre le comportement observé et le comportement attendu, une exigence, une spécification, un contrat ou une propriété de qualité.	Un défaut peut être fonctionnel, technique, documentaire, opérationnel, de performance, de sécurité ou de données.
Anomalie (anomaly)	Comportement, mesure ou observation inhabituelle qui mérite investigation, sans conclure immédiatement qu'il s'agit d'un défaut confirmé.	Une anomalie devient un défaut lorsqu'un écart attendu/observé est établi.
Incident (incident)	Événement en environnement réel ou opérationnel qui perturbe, dégrade ou empêche l'usage attendu d'un produit, service ou système.	Un incident peut révéler un défaut, une mauvaise configuration, une limite connue, une défaillance externe ou un problème d'usage.
Problème (problem)	Cause sous-jacente ou hypothèse de cause pouvant expliquer un ou plusieurs incidents ou défauts récurrents.	Traiter un problème vise à éliminer la cause, pas seulement à fermer l'incident.
Ticket de support (support ticket)	Enregistrement structuré d'une demande, question, incident, anomalie ou besoin d'assistance provenant d'un utilisateur, client ou intervenant interne.	Un ticket de support est une source de signal ; il doit être qualifié avant de devenir exigence, défaut ou action planifiée.
Sévérité (severity)	Mesure de l'impact réel ou potentiel d'un défaut, incident ou risque sur l'utilisateur, le client, les opérations, la sécurité, la conformité ou la livraison.	La sévérité décrit l'impact ; elle ne dicte pas seule l'ordre de traitement.

Priorité (priority)	Ordre relatif de traitement d'un élément selon valeur, urgence, risque, sévérité, dépendances, capacité et stratégie.	La priorité peut différer de la sévérité : un élément très sévère peut être rare ou mitigé, et un élément moins sévère peut bloquer une livraison.
Contournement (workaround)	Solution temporaire permettant de poursuivre l'usage ou de réduire l'impact sans corriger nécessairement la cause racine.	Un contournement n'est pas une correction permanente ; il doit être documenté avec ses limites.
Correctif (fix)	Changement visant à éliminer ou réduire un défaut, une cause d'incident ou un comportement non conforme.	Un correctif doit être validé par confirmation de correctif et, lorsque pertinent, par non-régression.
Analyse de cause racine (root cause analysis / RCA)	Investigation structurée visant à comprendre pourquoi un incident, défaut ou échec est survenu, afin d'éviter sa répétition.	La cause racine peut être technique, organisationnelle, documentaire, processuelle ou liée à une hypothèse erronée.
Post-mortem (post-mortem)	Revue structurée après incident ou échec significatif pour documenter faits, impacts, causes, apprentissages et actions d'amélioration.	Un bon post-mortem vise l'apprentissage sans blâme et produit des actions traçables.
Escalade (escalation)	Transfert ou élévation d'un sujet vers un niveau de compétence, d'autorité ou d'attention plus approprié.	Escalader ne veut pas dire se débarrasser du problème ; cela rend visible le besoin de décision, d'expertise ou de capacité.

Références

- SPÉCIFICATION

- Formation continue
- SDLC officiel
- Comment définir
- Contrat qualité officiel OPAL-RT
- Les tests (niveau × interface × angle × type)
- Exemples concrets de combinaisons
- Recommandations

Règle d’évolution du lexique

Lorsqu’un terme est ajouté ou modifié, il faut viser une définition stable, réutilisable et non contradictoire avec les pages de référence. Si un terme est ambigu, la définition doit inclure ce qu’il veut dire ici, ce qu’il ne veut pas dire, et les confusions fréquentes à éviter.